

Workshop No. 3 — Applying SOLID Principles to a Domotic Circuit Simulator

Object-Oriented Programming
Semester 2025-II
Universidad Nacional de Colombia

Authors:

Andres David Hincapié Álvarez
Samuel Alejandro Bulla Fiquitiva
Daniel Alejandro Clavijo Gonzalez

SOLID Principles Analysis:

(SRP)

The Single Responsibility Principle states that *a class should have only one reason to change*. In other words, each class must focus on a single, well-defined responsibility. When a class performs multiple tasks, it becomes harder to maintain and more prone to errors.

```
"error"
class Sensor:
    def read(self):
        pass
    def log(self, data):
        pass

"correct"
class Sensor:
    def read(self):
        pass

class Logger:
    def save(self, data):
        pass
```

In the corrected version, the Sensor only reads data, and the Logger is responsible for saving it.

(OCP)

The Open/Closed Principle states that *software entities should be open for extension but closed for modification*. This means that adding new functionality should not require changing existing, stable code. Instead, new behavior should be added through inheritance or composition.

```
"error"
class Sensor:
    def read(self):
        raise NotImplementedError

"correct"
class TemperatureSensor(Sensor):
    def read(self):
        return 25 # example value
```

To extend it with a new type of sensor, we do not modify the base class. We simply create a new subclass, The original sensor class remains unchanged, respecting OCP.

(LSP)

The Liskov Substitution Principle states that objects of a subclass should be usable in place of objects of the parent class without breaking the program. A subclass must behave in a way that is consistent with the expectations of its parent type.

```
"error"
class Sensor:
    def read(self):
        return 0

"correct"
class LightSensor(Sensor):
    def read(self):
        return 80 # valid replacement
```

Any function expecting a Sensor can safely accept a LightSensor because it maintains the same behavior expectations: it returns a numeric reading.

If a subclass returned a string or raised unexpected errors, it would violate LSP because it would no longer be usable as a true replacement for the parent class.

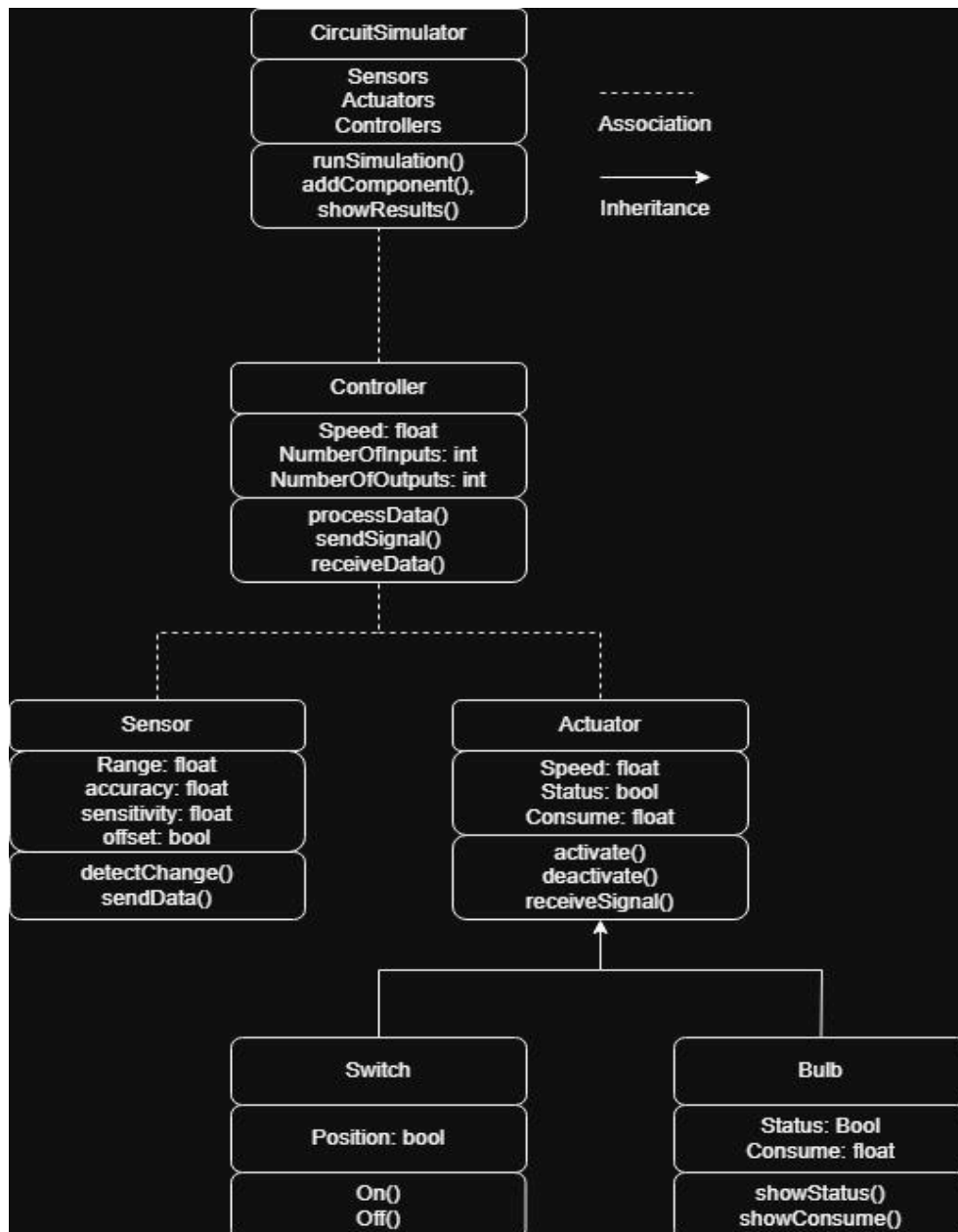
Updated UML and CRC Cards:

Updated CRC Cards:

Sensor	
Responsibility	Collaborator

Detect changes in the environment (movement, temperature, light). Send readings to a controller / processor. Activate something according to the measured values.	Controller Bulb Actuator
Controller	
Responsibility	Collaborator
Process system logic based on sensor data. Coordinate communication between sensors and actuators. Make decisions (interpret data, generate signals, manage component behavior).	Sensor Actuator
Actuator	
Responsibility	Collaborator
Perform physical actions. Receive commands from the controller. Confirm its status (on/off).	Controller Sensor
Simulador de Circuito	
Responsibility	Collaborator
Create and manage the system instances (sensors, actuators, controller). Run the simulation step-by-step. Display the circuit's results as visuals or text.	Sensor Actuator Controller User Interface
Bulb	
Responsibility	Collaborator
Turn on or off upon receiving a signal. Display its current state (on/off). Show energy consumption data.	Switch Controller Sensor
Switch	
Responsibility	Collaborator
Change its state (on/off). Send a signal to the bulb and/or controller.	Controller Bulb

Updated UML:



The diagram organizes the system into classes that represent the core components of the circuit simulator. Inheritance is applied in classes such as **Switch** and **Bulb**, which derive from **Actuator** and override methods like `activate()` for specific behaviors. Polymorphism enables the simulator to handle different actuators in a uniform way through their shared base class. Abstraction is reflected in classes like **CircuitSimulator**, which coordinates sensors, controllers, and actuators while hiding the internal logic and interactions from the rest of the system.

Python Code Snippets:

S — Single Responsibility Principle (SRP)

A class should have one responsibility.

If a class does too many things, it becomes harder to maintain.

```
1  class Sensor:
2      def __init__(self, initial_value=0):
3          self.value = initial_value
4
5      def read(self):
6          return self.value
7
8      def set_value(self, new_value):
9          self.value = new_value
10
11
12 class Logger:
13     def __init__(self):
14         self.memory = []
15
16     def save(self, data):
17         self.memory.append(data)
18         print("Saved:", data)
19
```

O — Open/Closed Principle (OCP)

Software should be open for extension but closed for modification.

You should be able to add new behavior without changing existing code.

```
class LightSensor(Sensor):
    def __init__(self):
        super().__init__(75)

class TemperatureSensor(Sensor):
    def __init__(self):
        super().__init__(28.5)
```

L — Liskov Substitution Principle (LSP)

Subclasses must be usable anywhere the parent class is expected.

```
def read_sensor(sensor: Sensor):  
    print("Reading:", sensor.read())  
  
read_sensor(LightSensor())  
read_sensor(TemperatureSensor())
```

I — Interface Segregation Principle (ISP)

Classes should not be forced to implement methods they don't need.
Use small, focused interfaces.

```
class Actuator:  
    def __init__(self):  
        self.state = False  
  
    def activate(self):  
        self.state = True  
  
    def deactivate(self):  
        self.state = False  
  
class Bulb(Actuator):  
    def activate(self):  
        super().activate()  
        print("Bulb ON")  
  
    def deactivate(self):  
        super().deactivate()  
        print("Bulb OFF")
```

D — Dependency Inversion Principle (DIP)

High-level modules should depend on abstractions, not concrete classes.

```
class Controller:
    def __init__(self, sensor: Sensor, actuator: Actuator):
        self.sensor = sensor
        self.actuator = actuator

    def update(self):
        value = self.sensor.read()
        print("Controller received:", value)

        if value > 50:
            self.actuator.activate()
        else:
            self.actuator.deactivate()
```

Reflection:

Applying SOLID principles to the domotic circuit simulator was a challenging experience. One of the main difficulties was learning to separate responsibilities correctly. At first, it was tempting to place multiple tasks inside a single class like mixing sensing, decision-making, and output actions because this looked easier. However, implementing the **Single Responsibility Principle** made us realize how dividing functionalities into focused, independent classes leads to cleaner and more maintainable code.

Understanding the **Open/Closed Principle** also presented a challenge. It required thinking ahead and designing classes that could be extended without modification. This meant avoiding rigid structures and instead relying on inheritance. Once achieved, it became clear how much easier it is to add new types of sensors or actuators with minimal changes to the existing code. Similarly, the **Liskov Substitution Principle** taught us to design subclasses that behave consistently with their parents, and we realized how violating this rule can break the logic of a program in unexpected ways.

Applying **SOLID principles** sometimes requires writing more classes and abstractions than initially expected, which can feel overwhelming. However, the long-term benefits: **scalability, readability, and avoiding duplicated logic**. Finally, the exercise reinforced the importance of modeling before coding. Creating CRC Cards and UML diagrams allowed us to visualize responsibilities and interactions more clearly, which made the implementation phase more intuitive.

Overall, this workshop helped us to understand that SOLID principles are not just theoretical rules but practical tools that significantly improve software design.